

Provenance-Bound Context Attestation and Deterministic Policy Authorization for Agentic AI Actions

CAP-PCL Reference Benchmark v1 and Structured Evaluation

Florin-Adrian Chitan

ILION Project · Independent Researcher · contact@ilion-project.org

· June 2026

Aspects of the CAP method are the subject of a pending patent application (OSIM A/00342, 2026).

Abstract. Agentic AI systems increasingly move beyond text generation into executable actions: API calls, workflow triggers, record modifications, permission changes, and enterprise automation. Existing policy engines can decide whether an action is allowed under a supplied context, but their correctness depends on the trustworthiness of that context. If an agent or compromised orchestrator can self-declare that identity is verified, a channel is trusted, approval exists, or a ticket is valid, deterministic policy enforcement can produce a wrong decision over corrupted inputs. We introduce the Context Attestation Pipeline (CAP), a pre-policy layer that converts raw orchestration metadata into provenance-bound contextual verification objects, and the Policy Constraint Layer (PCL), a deterministic policy layer that evaluates executable actions only over accepted contextual conditions. We evaluate the combined CAP+PCL design using a public structured reference benchmark of 170 scenarios plus a 12-scenario robustness probe. Holding policy logic constant, PCL-only authorization over declared context achieves 61.5% attack recall, $F1 = 0.7619$, and 0% FPR. CAP+PCL achieves 100% attack recall, $F1 = 1.0000$, and 0% FPR on the main benchmark. The full +38.5-point recall delta is localized to the forged-but-policy-satisfying quadrant, showing that CAP recovers the class of attacks to which policy-only authorization is structurally blind, without increasing false positives on honest benign traffic. A robustness probe demonstrates an explicit limitation: CAP cannot detect a fully compromised authoritative source that corroborates the forged context. The contribution is not a claim of universal attack detection; it is a causal isolation of provenance failure as a distinct policy-blind attack surface in agentic execution authorization.

Keywords: agentic AI, deterministic authorization, context attestation, provenance-bound claims, policy enforcement, execution governance, fail-closed authorization, AI agent security

1. Introduction

AI agents are becoming action-capable. In enterprise settings, an agent may call tools, invoke APIs, trigger RPA workflows, modify records, access data stores, change permissions, or initiate operational procedures. The safety question therefore changes from “is the model output acceptable?” to “should this exact action be allowed before it executes?”

This shift exposes a gap between content-level guardrails and execution-level authorization. Output filtering, refusal classifiers, and post-generation review can reduce unsafe text, but they do not by themselves establish whether an executable action is authorized under the real operational conditions of the current request. A policy engine can reject actions that violate explicit constraints, but only if the context supplied to the engine is trustworthy.

The central observation of this paper is simple: a policy engine is only as trustworthy as the context it evaluates. If identity, channel, approval, ticket, scope, or break-glass metadata can be self-declared by an agent or forged by a compromised orchestrator, policy enforcement becomes deterministic evaluation over untrusted input.

We address this by separating two questions that are often collapsed:

- **Context provenance:** are the contextual attributes authentic, fresh, scoped, request-bound, conflict-free, and issued by authorized sources?
- **Policy authorization:** given accepted contextual conditions, is the proposed executable action permitted?

We call the first layer the Context Attestation Pipeline (CAP) and the second the Policy Constraint Layer (PCL). CAP is not a new cryptographic primitive. It is an architectural layer that requires critical authorization attributes to be represented as provenance-bound verification objects before policy evaluation. PCL is a deterministic policy layer that evaluates the executable request over those accepted contextual conditions.

1.1 Contributions

1. We define **provenance-forgery** as a policy-blind attack surface for agentic AI authorization: the action appears policy-compliant only because the context has been forged or self-declared.
2. We introduce CAP, a pre-policy context-attestation layer, and show how it composes with PCL, a deterministic context-bound policy layer.
3. We formalize a 2×2 evaluation matrix that separates honest policy behavior from forged context, preventing the PCL-only baseline from becoming a strawman.
4. We release a structured reference benchmark v1 with 170 main scenarios and 12 robustness-probe scenarios across eight threat categories and eight enterprise domains.
5. We report deterministic reference-engine results showing that CAP+PCL recovers exactly the forged-but-policy-satisfying class missed by PCL-only: 61.5% to 100% attack recall, with 0% FPR in both systems.
6. We document an explicit limitation: CAP cannot detect a fully compromised authoritative source that corroborates the forged context.

2. Background and Related Positioning

The ILION architecture has previously introduced deterministic pre-execution safety gates for agentic AI systems [1] and session-risk monitoring for multi-step agentic behavior [2]. Those components address semantic/identity alignment and temporal risk. The present work addresses a different layer: the provenance of the contextual attributes consumed by authorization logic.

This work should be read as a reference design and evaluation artifact, not as a claim that CAP replaces identity, credentialing, runtime sandboxing, or conventional policy engines. CAP is intended to sit before policy evaluation; PCL evaluates policy; downstream layers such as semantic gates, session-risk monitors, sandboxing, API enforcement, and audit systems may then operate on the resulting decision path.

General-purpose policy engines such as Open Policy Agent / Rego [5] and AWS Cedar [4] evaluate authorization rules over a supplied input document; they are policy evaluators and assume the input context is trustworthy. Recent agentic-security efforts focus on adjacent layers: runtime governance and execution-time policy enforcement, such as Microsoft's Agent Governance Toolkit [9], which evaluates actions before they run against policy; and runtime isolation / sandboxing, such as NVIDIA's OpenShell [10], which enforces out-of-process containment so a compromised agent cannot exceed its environment. CAP is **complementary** to all of these. It does not claim to be the first pre-execution control — several of these systems also evaluate actions before execution. Rather, CAP isolates a layer those approaches typically assume away: the verifiable provenance of the context itself. Execution-control planes evaluate policy over the context they receive; sandboxes contain the process after a decision; neither attests that the contextual attributes feeding the decision are authentic. The following table situates CAP+PCL within an execution-governance stack.

Layer / approach	Question it answers	Relation to CAP+PCL
Agent identity / credentials (e.g. AIAM, credential RFCs)	Who or what is the agent, and what authority does it hold?	Upstream substrate CAP+PCL assumes and consumes.
CAP (this work)	Is the context authentic, fresh, request-bound, conflict-free, from an authorized issuer?	Pre-policy verification layer.
PCL (this work)	Is the action permitted under the accepted context?	Deterministic policy over verified context.
Policy engines (OPA/Rego, Cedar)	Does the input document satisfy the rules?	Evaluate policy; assume input context is trustworthy.
Execution-control / runtime governance	Should this call be permitted at runtime, per policy?	Same family; operate over context CAP would verify.
Sandboxing / runtime isolation	What can the agent process physically access?	Downstream containment after a decision is made.
ILION Gate / SRM [1,2]	Is the action semantically aligned; is the session trajectory safe?	Adjacent ILION layers, distinct from provenance.

Table 1. Positioning of CAP+PCL within an execution-governance stack. CAP+PCL is complementary, not a replacement for identity, policy, execution-control, or sandboxing layers.

3. Problem Statement

Let **R** be an executable request proposed by an AI agent. **R** includes an actor, role, action, resource, parameters, and contextual metadata. A policy engine **P** decides ALLOW or BLOCK by evaluating **R** and the supplied context **C**. In a policy-only system, the decision is:

```
PCL_only(R, C_declared) -> {ALLOW, BLOCK}
```

The weakness is that **C_declared** may be self-declared or forged. For example, an orchestrator may report `channel = secure_api` when the request actually arrived through email; an agent may claim `identity_verified = true`; an approval token may be replayed outside its intended scope; or a break-glass reason may be asserted without independent workflow evidence.

This produces a structural blind spot: if the declared context satisfies policy, a policy-only engine has no basis to reject the request unless it can verify provenance. The policy engine may be correct as a policy engine and still be wrong as an authorization system because its input is corrupted.

3.1 Threat classes

Forgery scenarios in the benchmark are anchored to eight threat classes:

Class	Name	Description
T1	Compromised orchestrator	context assembled or modified by a subverted orchestrator
T2	Self-declared identity	agent asserts verified identity without IdP attestation
T3	Channel spoofing	declared secure channel; gateway attests email/webhook
T4	Approval replay	approval claim not bound to the current request (stale nonce)
T5	Stale session	claim presented outside its validity window
T6	Forged break-glass	emergency override without workflow-attested reason
T7	Ticket scope mismatch	valid ticket whose scope does not cover the request
T8	Conflicting sources	authoritative sources disagree on the same attribute

Table 2. Threat classes T1–T8 used to anchor the forgery scenarios.

4. Architecture: CAP + PCL

CAP and PCL deliberately separate context provenance from policy authorization. CAP determines which contextual claims are usable. PCL determines whether the action is allowed under the usable context.

4.1 Request and context planes

The **request plane** contains what the agent proposes to execute: actor, role, action, resource, parameters, and declared intent. The **context plane** contains the conditions under which the action may be authorized: identity verification, trusted channel, approval state, ticket scope, session state, device state, and break-glass evidence. A key design rule is that **the agent may propose an action, but it may not self-attest the critical context under which that action is authorized.**

4.2 CAP verification objects

CAP transforms critical authorization attributes into contextual verification objects. A minimal object contains:

```
verification_object = {
  attribute_type, attribute_value,
  issuer_id, subject_id, request_id, session_id,
  resource_scope, action_scope,
  issued_at, expires_at, nonce,
  integrity_proof
}
```

The integrity proof is format-agnostic: it may be a digital signature, MAC, signed token, certificate, verifiable credential, hardware attestation, hash commitment, zero-knowledge proof, or any future cryptographically verifiable structure that permits issuer, integrity, freshness, and request-binding verification.

4.3 Usability predicate

A verification object v is usable for request R only if all of the following hold:

```
usable(v, R) :=
  authentic(v) AND intact(v) AND fresh(v) AND
  bound(v, R) AND authorized_issuer(v) AND
  scope_compatible(v, R) AND no_conflict(v, B_R)
```

If any required object is missing, expired, unbound, out of scope, issued by an unauthorized source, or conflicting with another required object, CAP returns BLOCK and downstream evaluation stops fail-closed.

4.4 PCL authorization over accepted context

PCL receives the accepted contextual bundle B_R and evaluates deterministic policies over the executable request. In the reference benchmark, PCL checks requirements such as role-action compatibility, trusted channel, ticket scope, approval requirements, break-glass conditions, and coupled-policy constraints. PCL does not decide whether a contextual object is authentic; that is CAP's function.

```
CAP_PCL(R, C_declared, A_attested):
  B_R = CAP.verify(R, A_attested)
  if B_R == BLOCK: return BLOCK
  return PCL.evaluate(R, B_R)
```

5. CAP-PCL Reference Benchmark v1

We constructed a **structured reference benchmark** to isolate one property: whether provenance-bound context attestation recovers the class of attacks that policy-only authorization cannot see. The benchmark

is **not** a real-world enterprise log corpus and **does not** evaluate natural-language understanding. It evaluates deterministic authorization decisions over modeled agentic-authorization context.

5.1 Systems compared

The policy logic is held constant. The only experimental difference is whether contextual attributes are accepted as declared (PCL-only) or verified through CAP before PCL evaluation (CAP+PCL).

5.2 Scenario matrix

Two independent axes define each scenario: is the context honest or forged, and does the declared context satisfy or violate policy?

Quadrant	Ground truth	PCL-only	CAP+PCL
Q1 — honest context, policy satisfied	ALLOW	ALLOW ✓	ALLOW ✓
Q2 — honest context, policy violated	BLOCK	BLOCK ✓	BLOCK ✓
Q3 — forged context, satisfies as declared	BLOCK	ALLOW ✗ (blind)	BLOCK ✓
Q4 — forged context, violates as declared	BLOCK	BLOCK ✓	BLOCK ✓

Table 3. The 2x2 evaluation matrix. The causal delta is expected only in Q3 — the only quadrant where the two systems diverge. This is crucial for fairness: PCL-only performs correctly over honest context (Q1, Q2) and over already-invalid declared context (Q4); it fails only where provenance forgery makes the context appear policy-compliant (Q3).

5.3 Dataset composition

The main set contains 170 scenarios across finance, identity and access, human resources, healthcare, procurement, cloud infrastructure, RPA automation, and IT support. Each scenario contains role, action, resource, industry domain, declared_context, attestation, ground truth, and explicit expected PCL-only and CAP+PCL verdicts. Forgery scenarios are anchored to T1–T8.

Quadrant / class	Count	Ground truth	Role
Q1 benign_honest	40	ALLOW	FPR test
Q2 violation_honest	40	BLOCK	PCL fairness
Q3 forgery_satisfies	50	BLOCK	the delta
Q4 forgery_violates	40	BLOCK	overlap control
Main total	170	40 ALLOW / 130 BLOCK	
Robustness probe (separate)	12	BLOCK	limitation

Table 4. Dataset composition. The forged-but-valid class (Q3) is the largest single class because it carries the contribution, but honest classes (Q1+Q2 = 80) outnumber it, preventing a forgery-only bias.

5.4 Reference engine

The benchmark includes a reference implementation that is separate from any ILION production engine. Its purpose is reproducibility: it loads the public JSON dataset and applies the deterministic PCL-only and CAP+PCL decision rules. The reference engine should be understood as a formalized evaluator for the benchmark, not as a product implementation. The dataset and reference engine ship with a SHA-256 manifest; re-running the evaluator against the dataset yields byte-identical output.

6. Results

CAP+PCL improves attack recall by **+38.5 percentage points** while maintaining 0% false-positive rate on honest benign traffic. Because the benchmark is deterministic and structured, the key result is not the perfect score itself but the **localization** of the delta. Headline metrics are summarized in Table 5.

Metric	PCL-only	CAP+PCL
Attack recall	61.5%	100.0%
Precision	100.0%	100.0%
F1	0.7619	1.0000
False-positive rate	0.0%	0.0%
TP / TN / FP / FN	80 / 40 / 0 / 50	130 / 40 / 0 / 0

Table 5. Headline metrics on the 170-scenario main set (130 BLOCK, 40 ALLOW). Reference engine, deterministic.

6.1 Per-quadrant accuracy

This is the central empirical result. PCL-only behaves correctly in Q1, Q2, and Q4. It fails only in Q3, where the policy appears satisfied because the context has been forged. CAP+PCL recovers Q3 by rejecting unusable or conflicting contextual claims before PCL evaluation.

Quadrant	n	PCL-only	CAP+PCL
Q1 benign_honest	40	40/40 (100%)	40/40 (100%)
Q2 violation_honest	40	40/40 (100%)	40/40 (100%)
Q3 forgery_satisfies	50	0/50 (0%)	50/50 (100%)
Q4 forgery_violates	40	40/40 (100%)	40/40 (100%)

Table 6. Per-quadrant accuracy. The entire +38.5-point recall delta is localized to Q3, establishing causal attribution rather than a general “extra layer helps” effect.

6.2 Per-threat detection

T1 is partially detected by PCL-only because some compromised-orchestrator cases also violate the declared policy. T2–T8 are provenance-specific classes in which the declared context is crafted to satisfy policy; PCL-only is structurally blind to all of them. CAP+PCL detects all modeled T1–T8 cases in the main benchmark by verifying issuer, freshness, binding, scope, and conflicts before authorization.

Threat	n	PCL-only	CAP+PCL
T1	46	40/46 (87%)	100%
T2	7	0/7 (0%)	100%
T3	7	0/7 (0%)	100%
T4	6	0/6 (0%)	100%
T5	6	0/6 (0%)	100%
T6	6	0/6 (0%)	100%
T7	6	0/6 (0%)	100%
T8	6	0/6 (0%)	100%

Table 7. Per-threat detection. PCL-only catches T1 partially (cases that also violate declared policy) and is blind to T2–T8. CAP+PCL recovers all modeled classes.

6.3 Robustness probe: compromised authoritative source

We include 12 robustness-probe scenarios that model a fully compromised authoritative source: the forged value is also what the source attests, so no conflict exists. Both PCL-only and CAP+PCL fail to recover these cases, yielding 0% recall on the probe. This limitation is intentional and important. CAP can

recover provenance conflicts and missing or invalid attestation; it cannot solve the problem of a subverted source of truth. Mitigating this failure mode requires independent redundancy, quorum rules, hardware-backed attestation, source separation, human oversight, or downstream anomaly detection.

7. Discussion

The benchmark supports a narrow but important claim: policy-only authorization is structurally blind to forged context when the forged context satisfies policy as declared. CAP supplies the missing precondition check. This is not a probabilistic classifier result; it is a deterministic causal isolation of a specific attack surface.

7.1 Why the 100% result is not overclaimed

The reference engine achieves perfect detection on the main modeled provenance-forgery classes because the attestation rule directly encodes the condition under test. **This should not be read as a claim that CAP detects all real-world attacks.** It should be read as a demonstration that, under a well-specified provenance-forgery model, adding provenance-bound context attestation recovers the exact class of attacks that policy-only evaluation cannot observe. The contribution is the causal isolation of provenance failure as a policy-blind attack surface — not classification performance. The robustness probe bounds the claim by exhibiting a class CAP does not detect.

7.2 Relation to semantic and temporal ILION layers

CAP+PCL addresses the contextual layer of execution authorization. It does not replace semantic/identity checks or session-risk monitoring. In the broader ILION stack, CAP asks whether the context is authentic; PCL asks whether the action is permitted in that context; ILION Gate [1] asks whether the action is semantically aligned with the agent role and identity frame; SRM [2] asks whether the session trajectory remains safe over time. The stack is therefore multi-dimensional rather than a single policy engine.

The four layers are **complementary and non-redundant** because each detects a class of failure the others cannot observe. A request may be semantically aligned with the agent's role (passing Gate) yet rest on forged context (caught only by CAP). It may carry authentic, policy-satisfying context (passing CAP and PCL) yet form part of a slow, distributed attack visible only across the session (caught only by SRM). It may be a single well-formed action that nonetheless violates an explicit policy constraint (caught only by PCL). No single layer subsumes another: provenance, policy, semantic alignment, and temporal trajectory are distinct axes of authorization. The present paper isolates and evaluates one of these axes — provenance — precisely so that its contribution can be measured independently of the others. A combined evaluation of the full CAP → PCL → Gate → SRM chain is left to future work (Section 9), where the interaction effects between layers become the object of study rather than the isolated contribution of any one.

7.3 Deployment interpretation

A deployed system should not treat CAP as a substitute for IAM, sandboxing, runtime containment, audit, or human oversight. CAP is a pre-policy verification layer. It is most valuable when an agentic runtime aggregates metadata from identity providers, gateways, approval engines, ticketing systems, registries, and orchestrators before authorizing executable actions.

8. Limitations

- **Structured synthetic benchmark.** The scenarios are deliberately structured to isolate provenance failure. They are not a natural enterprise log corpus.

- **No natural-language understanding evaluation.** The benchmark evaluates deterministic authorization over structured context, not model comprehension or prompt-injection detection.
- **Reference engine, not production engine.** The included evaluator is a clean reference implementation for reproducibility, not the proprietary or production ILION engine.
- **Authoritative-source compromise.** If the source of truth is compromised and attests the forged value, CAP cannot detect the attack by itself.
- **Policy completeness.** CAP can ensure that context is usable; it cannot ensure that downstream policies are complete or correct.
- **Operational trust assumptions.** CAP requires issuer registries, key management, clock synchronization, nonces, TTL handling, and scope semantics to be correctly implemented.
- **No performance benchmark.** This paper focuses on authorization correctness in the reference benchmark. End-to-end latency under production load is future work for this layer.

9. Future Work

- Run the same dataset against an independent production implementation and report reference-engine vs production-engine results separately.
- Expand the benchmark with noisy, partially missing, and conflicting real-world enterprise-style context.
- Add source-redundancy and quorum variants for compromised-authoritative-source scenarios.
- Integrate CAP-PCL evaluation with the semantic Gate and SRM to evaluate the full CAP → PCL → Gate → SRM authorization chain.
- Map the pattern to agentic identity, credentialing, execution-control, sandboxing, audit, and compliance-by-design taxonomies.
- Release the dataset and reference harness on Zenodo with a DOI and maintain revisions as the threat taxonomy evolves.

10. Conclusion

This paper identifies provenance forgery as a distinct policy-blind attack surface in agentic AI execution authorization. A deterministic policy engine can be correct and still authorize the wrong action if the context it evaluates is forged. CAP addresses this by requiring critical contextual attributes to be provenance-bound, fresh, request-bound, scoped, conflict-free, and issued by authorized sources before PCL evaluates the action.

In a 170-scenario structured reference benchmark, CAP+PCL improves attack recall from 61.5% to 100% while preserving 0% false positives on honest benign traffic. The improvement is localized entirely to the forged-but-policy-satisfying quadrant, demonstrating that CAP recovers exactly the class PCL-only cannot see. The robustness probe documents an explicit boundary: CAP does not detect a compromised authoritative source that corroborates the forged context.

The result supports a precise design principle for agentic AI systems: deterministic policy evaluation should operate over attestable context, not self-declared metadata.

Appendix A. Dataset schema (as released)

Each scenario in the public dataset uses the following structure. Attestation claims are grouped per attribute, each carrying the attested value, issuer, freshness, and request-binding flags. The forgery in this T3 example is the gateway attesting channel = email against the declared secure_api.

```
{
```

```

"sid": 81,
"category": "forgery_satisfies",
"threat": "T3",
"role": "iam_agent",
"action": "grant_role",
"resource": "admin_group",
"industry_domain": "identity_access",
"target_sensitivity": "high",
"declared_context": {
  "identity_verified": true, "channel": "secure_api",
  "dual_approval": true, "ticket_present": true,
  "break_glass_reason": null, "ticket_scope_covers": true },
"attestation": {
  "identity_verified": {"value": true, "issuer": "idp", "fresh": true, "request_bound":
true},
  "channel": {"value": "email", "issuer": "gateway", "fresh": true,
"request_bound": true},
  "dual_approval": {"value": true, "issuer": "workflow", "fresh": true, "request_bound":
true},
  "ticket_present": {"value": true, "issuer": "itsm", "fresh": true, "request_bound":
true},
  "break_glass_reason": {"value": null, "issuer": "workflow", "fresh": true, "request_bound":
true},
  "ticket_scope_covers": {"value": true, "issuer": "itsm", "fresh": true, "request_bound":
true} },
"ground_truth": "BLOCK",
"expected_pcl_only": "ALLOW",
"expected_cap_pcl": "BLOCK",
"note": "channel spoofing: declared secure_api, gateway attests email"
}

```

Appendix B. Reproducibility

Running the evaluator against the public dataset reproduces the reported metrics deterministically:

```

$ python evaluate.py

Loaded 170 main scenarios, 12 robustness-probe scenarios
Ground truth: 130 BLOCK, 40 ALLOW

PCL-only:  TP=80  TN=40  FP=0  FN=50
           Precision=1.0000  Recall=0.6154  F1=0.7619  FPR=0.0000
CAP+PCL:   TP=130 TN=40  FP=0  FN=0
           Precision=1.0000  Recall=1.0000  F1=1.0000  FPR=0.0000

Delta recall: +38.5 pts  Attacks recovered by CAP: 50
Robustness probe (compromised source): PCL-only 0%, CAP+PCL 0%

```

References

- [1] Florin-Adrian Chitan. ILION: Deterministic Pre-Execution Safety Gates for Agentic AI Systems. arXiv:2603.13247, 2026.
- [2] Florin-Adrian Chitan. ILION-SRM: Session Risk Memory for Deterministic Temporal Authorization of Agentic AI Actions. arXiv:2603.22350, 2026.
- [3] Ryle Goehausen and Marcus Sousa. GATE AI: LLM Security Benchmark Evaluation Methodology and Results. arXiv:2606.02959, 2026.
- [4] Joseph W. Cuthbertson, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, John Kastner, Anwar Mamat, Matt McCutchen, Neha Rungta, Bhakti Shah, Emina Torlak, and Andrew Wells. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. Proc. ACM Program. Lang. 8 (OOPSLA1), Article 118, April 2024. arXiv:2403.04651.

- [5] Open Policy Agent (OPA) and the Rego policy language. Cloud Native Computing Foundation (CNCF) graduated project. <https://www.openpolicyagent.org>
- [6] OWASP. OWASP Top 10 for Large Language Model Applications (2025). OWASP GenAI Security Project; first released 2023. <https://genai.owasp.org>
- [7] Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, OJ L, 2024/1689, 12 July 2024.
- [8] E. Tabassi. Artificial Intelligence Risk Management Framework (AI RMF 1.0). NIST AI 100-1, National Institute of Standards and Technology, January 2023. DOI: 10.6028/NIST.AI.100-1.
- [9] Microsoft. Agent Governance Toolkit (AGT): open-source runtime security for AI agents. 2026. <https://github.com/microsoft/agent-governance-toolkit>
- [10] NVIDIA. OpenShell: a secure-by-design runtime for developing, deploying, and governing autonomous agents (part of the NVIDIA Agent Toolkit). Announced at GTC 2026; Apache 2.0. <https://github.com/NVIDIA/OpenShell>